

Honest feedback on product viability

Shared by SAMBHAV JAIN

This is a copy of a chat between Claude and SAMBHAV JAIN. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. [Report](#)

Files hidden in
shared chats



be real with the see my whoole idea and see how we will built it and abd
review the whole project and no sugar coating be real tell me exactly tht do
u think peoppe will buy it

Searched the web

Okay, real talk, no cushioning.

The core problem: this exact product already exists, is live, free, and built by a much better-resourced team.

Cognition Labs (the Devin team) launched **DeepWiki** in April 2025. It does the large majority of what you've spec'd: point it at any GitHub repo, it builds a structural understanding model, generates architecture diagrams, module-level explanations, dependency maps, and a conversational AI assistant that answers questions grounded in the actual source. It's pre-indexed over 50,000 public repos, reportedly spending more than \$300,000 in compute to process over 4 billion lines of code. It ships language-specific diagrams — for a React project it identifies the component tree and data flow, for a Python microservice it maps the request lifecycle from route handler through middleware to the database. It even has a scoped "deep research" mode for security/migration analysis, and as of last month it ships an **MCP server** so Cursor/Claude Code/Windsurf can pull this context directly into an agent's workflow instead of a human browsing a graph. It's free for public repos, and Cognition has already run it at enterprise scale — one banking client used the private version to document over 400,000 internal repos. [Codersera](#) [Codersera](#)

That's not "a competitor exists." That's "the specific wedge you're describing — repo in, understanding model out, diagrams + AI chat on top — is already solved, funded at a level you can't match, and being pushed toward the exact place your users actually spend time (their coding agent), not toward a new dashboard they have to visit."

Second data point, and it's an uncomfortable one: this idea has been tried before, specifically.

CodeSee (2019–2024) pitched itself as literally "Google Maps for code" — automated maps of your codebase, dependency visualization, onboarding acceleration, business logic layered onto the graph. It raised real money, had SOC2 certification, enterprise customers, years of iteration. It did not become a breakout company. It got quietly folded into GitKraken as a feature. That's the outcome for a well-funded team executing your exact "graph as the interface" thesis for five years. That's a strong signal, not proof of impossibility, but you should sit with it before assuming the market wants this as a standalone platform.

The hard part of your doc is the part you've hand-waved.

The "Understanding Model" — parse arbitrary repos across languages/frameworks into business workflows, technical workflows, DB relationships, trust boundaries, auth systems, etc. — is not an MVP feature. That's the multi-year applied research problem DeepWiki spent real money solving, and even they say it's imperfect: "It is not a replacement for reading code — for anything business-critical, verify the AI's claims against the source." Real codebases are messy — dynamic typing, unconventional patterns, monorepos. Your doc's login-flow example is clean because it's a toy example. As a solo build in a college break, this piece alone is not "included in MVP," it's the whole company. Codersera

Who's actually buying this?

Founders and PMs don't open codebase graphs — they ask an engineer, or now, they ask Claude/ChatGPT. Developers already have Cursor, Claude Code, Copilot chat, and now DeepWiki's MCP sitting inside their existing tools. You'd be asking people to leave their workflow and visit a new app for something a plugin already does. And the buyer with real budget — enterprises doing onboarding/audits — needs to upload their entire proprietary codebase to a third party they've never heard of. That's a security review and procurement cycle, not a signup flow, and it's the same trust bar CodeSee had to clear with SOC2 for years.

What's actually good here, so this isn't just teardown:

The lens system (Business / Backend / Security) and per-node contextual AI chat is a genuinely nicer *interaction model* than DeepWiki's static wiki-per-page approach — DeepWiki explains a repo, your idea lets someone navigate it interactively at the exact abstraction level they care about. And the Security lens has real synergy with Sentinel — trust boundaries, attack surface mapping — that's a place you'd have actual differentiated expertise instead of competing head-on with a Devin-funded team.

Honest verdict: as scoped — a broad standalone platform competing on "understand any repo" — no, I don't think people pay for this, because the free, better-funded alternative already does the core job and is moving toward agent integration rather than a new UI to check. If you want this to survive contact with reality, shrink it hard: pick the one lens nobody else is doing well (security/trust-boundary mapping, tied to Sentinel) or build it as a plugin/MCP tool that feeds *existing* agent workflows instead of a destination app, rather than trying to out-build Cognition's understanding-model pipeline from scratch.

is deepwiki used like does it have a bigger user base

Searched the web

Good, found actual numbers. Here's the real picture.

Yes, it's used, and at a scale that matters.

Per Semrush's traffic data on deepwiki.com: in September 2025 deepwiki.com received 903.05K visits with an average session duration of 15:01. That's not a side project number — that's a genuinely used product, and a 15-minute average session is long for a dev tool, meaning people aren't bouncing, they're actually reading/using the wiki. Traffic held steady month over month — 949K in August, 929K in July, 903K in September, so it's not a launch spike that died, it's sustained usage nearly a year and a half after launch (launched April 2025). [Similarweb](#) [Similarweb](#)

Where that traffic comes from is also telling: on deepwiki.com, visitors mainly come from Direct traffic (60.15%), followed by github.com (7.71%). Direct traffic that high usually means people bookmark it or type it in — i.e., repeat/habitual usage, not one-off curiosity clicks from a blog post. [Similarweb](#)

Beyond web visits, the adoption signals from Cognition's own numbers back this up: DeepWiki has been used to document repositories ranging from small open-source utilities to massive enterprise codebases, and one banking client used the private version (Devin Wiki) to generate documentation across more than 400,000 internal repositories. And on the developer-tooling side specifically — the part that matters most for your use case — Cognition publishes an official MCP server at mcp.deepwiki.com that exposes DeepWiki's repo knowledge as tools to any MCP-compatible client, so agents like Claude Code, Cursor, and Windsurf can fetch a repo's wiki structure, pull a specific page as Markdown, or ask a natural-language question and get a cited answer directly inside the coding workflow. [Zen MCP Server](#) [Agentwiki](#)

What this means for you concretely:

- ~900K monthly visits + a free MCP server wired into Cursor/Claude Code/Windsurf means the exact audience you want (developers understanding unfamiliar repos) already has a free, sticky habit formed around a competitor with a nine-figure-funded parent company behind it.
- The "direct traffic" pattern tells you this isn't a novelty — people are coming back. That's the hardest thing to displace, not the feature list.
- Their model is also expanding, not standing still — they went from static wiki → conversational Q&A → MCP integration into agents in about a year. They're moving toward exactly where you'd want to compete (agent-embedded context), which narrows your window further.

This doesn't kill your idea, but it does mean "repo → understanding model → diagrams → chat" as a general-purpose destination product is competing against something with real, growing, habitual usage — not vaporware you can quietly outflank. If you go forward, the lens system and security-angle differentiation I mentioned earlier are still your best shot, but go in eyes open: you're not entering an empty market, you're entering one with an established, well-used incumbent.

it is completely free ?

Mostly, with a real catch.

Free tier: Public GitHub repos are free, no signup required. It's free for public repositories — you don't need to install anything or own the repo. Access to private repositories requires registration/authentication. Free public access covers over 50,000 pre-indexed repos plus any public repo you submit for indexing.

Where it's not free: Private repos require a Devin account — meaning DeepWiki for your own proprietary codebase (like Sentinel or a client project) is gated behind Cognition's paid Devin product, not the free public tool. Devin itself is a paid AI-engineer product (ACU/credit-based pricing), so if you wanted this for private code, you're paying for Devin access, not a separate DeepWiki subscription.

Also worth knowing: the free version has real limits — Fast vs Deep Research modes, and large repos may hit generation-size limits requiring a `.devin/wiki.json` config to steer what gets documented. And it's a snapshot, not live — wikis are regenerated on a schedule,

not on every commit, so for very active repos the wiki can lag main by hours to days (Devin users can manually re-trigger a refresh; free public users can't).

So the honest framing: for public/open-source repos, yes, fully free forever with no catch — that's exactly the segment you'd have the hardest time monetizing against them anyway. For private/enterprise repos — the segment with actual budget — it's not free, it's a wedge into Cognition's paid Devin ecosystem. That's actually the more useful data point for you: their free tier is a funnel toward a paid product, not charity. If you're thinking about where a wallet-open customer would spend money, it's the same segment they're already funneling toward Devin, not an open lane they've left uncovered.

know i mean if i have a repo to ubersrtna dit readi it. is free?

Depends on whether your repo is public or private on GitHub.

If it's a public repo — yes, completely free, no signup needed. Just swap "github.com" with "deepwiki.com" in the URL (e.g. `github.com/owner/repo` → `deepwiki.com/owner/repo`), and if it's already indexed you get the wiki, diagrams, and Q&A instantly. If it's not indexed yet, you can submit it for free indexing.

If it's a private repo (like your Sentinel repo, or client work) — no, not free. Private repos are only accessible through Devin's paid platform, since DeepWiki auto-indexes connected repos as part of onboarding to Devin. You'd need a paid Devin account to point it at your own private code.

So for reading someone else's open-source project to understand it — totally free, unlimited. For understanding your own private codebase — you'd be paying for Devin, not DeepWiki specifically.

Files hidden in
shared chats



basically this was my main idea the unsesting code like bloomberterminanl was to just get statrd and enter the market revei the ide idea and be real no sugar coating just facts

Searched the web

Alright, same honesty as before, now for the bigger vision.

The "Software Engineering OS" pitch is the single most common vision statement in this exact space right now — you're not the only one saying it, everyone is.

Look at GitHub's own language when they launched Copilot Workspace in 2024: "we are redefining the developer environment... where any developer can go from idea, to code, to software all in natural language." That's almost word-for-word your framing — understand, design, build, deploy, maintain, all in one platform. Cognition is explicitly building toward the same thing with Devin. Replit, Vercel, and a long list of funded startups are running this identical playbook. When you write "the entire software lifecycle should exist inside one intelligent platform," you're not describing a novel insight — you're writing the default ambition template every AI-dev-tool founder writes in their deck's vision slide right now. That doesn't make it wrong, but it means it won't read as differentiated to anyone who's seen more than a handful of these decks.

And here's the part that actually matters: the best-funded team already tried the literal version of this and had to walk it back.

GitHub Copilot Workspace launched April 2024 as exactly this — a browser-based environment where you hand it an issue and it plans, builds, tests, and runs code, replacing the fragmented toolchain with one natural-language-driven environment. By May 30, 2025, that technical preview was officially sunset. It wasn't abandoned — the lessons were rebuilt into the Copilot Coding Agent, GA since September 2025 — but notice the shape of what survived: not a new unified "OS" that replaces your dev environment, but an agent that plugs into the tools people already use (VS Code, existing repos, existing PR workflow), coordinated through things like Mission Control for managing multiple agents. Microsoft, with effectively unlimited capital, tried "one platform for the whole lifecycle" and the thing that actually stuck was "composable agent that meets developers where they already are," not the unified platform.

Cognition's own trajectory shows the same pattern: DeepWiki isn't trying to pull developers into a new home base, they're pushing outward via an MCP server so Cursor, Claude Code, and Windsurf can pull DeepWiki's understanding directly into the tools people already use. Both of the most credible players in this space are converging on "inject capability into existing workflows via protocol," not "build the new terminal everyone lives in."

The Bloomberg Terminal analogy doesn't structurally hold, and it's worth being blunt about why.

Bloomberg Terminal works because of things that don't have a software-engineering equivalent: a proprietary real-time data moat nobody else can license, a closed messaging network where your counterparties also have to be on the terminal to reach you, and a customer base that pays \$2k/month because the terminal's output directly makes or saves them money in the same session. Software engineering has none of that. There's no proprietary data — the code is the user's own. There's no network effect forcing collaborators onto your platform — git, CI, and cloud APIs are open standards specifically so tools stay swappable. And the entire current trend (MCP, agent tool-calling) is developers explicitly rejecting closed vertical suites in favor of composable tools that plug into whatever they're already using. "Terminal for X" is a great pitch-deck metaphor; it's a poor structural fit here.

Scope reality check, stated plainly: every item in your "future modules" list — requirements management, planning, system design, architecture design, database design, testing, deployment, monitoring, observability — is already its own multi-billion-dollar category with an entrenched leader (Linear/Jira, Figma/Miro, dbdiagram, Vercel/Render, Datadog/Grafana). No one — not GitHub with Microsoft behind it, not Atlassian, not Cognition — has actually unified these into one dominant platform despite years and billions of dollars of attempts. That's not a reason you can't try. It's a reason to know exactly what kind of fight this is before you write it into a v1 roadmap.

Straight answer: as a long-term vision paragraph in a deck, this is fine — everyone writes one, nobody expects you to have solved it yet, and judges/investors read past it to the wedge. As something you actually build toward, no, I don't think this specific framing wins, because the two most credible competitors in the exact wedge you'd start with (DeepWiki/Devin and GitHub Copilot) have already tested "build the unified platform" and both moved away from it toward "compose into existing tools." If you keep the OS narrative in your pitch as aspiration, that's normal and fine — just don't let it pull your actual v1 engineering effort away from the one narrow, defensible thing you have that they don't build well: the security/trust-boundary lens tied to Sentinel.

see basically i want to create a caodinf editor whene we can type code simply and insdead of the just a agent beside which halusunaetes so Which hallucinates sometimes, and does everything by its own. So taking a track of what is done and how it's done is difficult using these agents. For example, I told a agent to perform some changes, like, I gave gave it a simple plain prompt, like, move the button to more right or left position. What it did in mv code is it overlapped the code. It had some batch word. I told it to

[Show more](#)

Searched the web

Yes, I get exactly what you're saying — and I want to tell you something important before the critique: **you just independently re-derived a real, named category that the industry is currently calling "spec-driven development" (SDD).** Your diagnosis is correct, and it's not a niche observation — it's the exact failure mode the whole industry is currently trying to fix.

What you described has a name, and it emerged for exactly the reasons you gave.

The term "vibe coding" — accepting AI-generated code without reviewing or documenting it — became Collins Dictionary's word of the year for 2025, specifically because of the pattern you're describing: agent does a "quick fix" that actually patches over the problem instead of solving it (your disable-instead-of-remove example is the textbook case), nobody can trace what decisions were made or why, and every new session starts from zero because there's no persistent memory of what's already been decided. SDD is a methodology where a written specification — requirements, design, tasks — is treated as the primary artifact and code becomes the regenerable output, precisely to stop that drift and give both human and agent something durable to check work against.

And it's already a crowded, well-built category — not a gap. As of 2026, every major AI coding tool has shipped its own version:

- **AWS Kiro** — launched May 2025, forces requirements → design → tasks before any code is written, uses aerospace-grade EARS notation for machine-parseable specs, has "steering files" for persistent project context so you stop re-explaining yourself every session, "agent hooks" that auto-run on file save, and a checkpoint system to roll back to any prior state.
- **GitHub Spec Kit** — open source, 93,000+ GitHub stars, a Spec → Plan → Tasks → Implement pipeline that works across 30+ different agents (Copilot, Claude, Codex, Kiro, etc.), so you're not locked to one vendor.
- **OpenSpec** — 52,100 stars, enforces a strict proposal → apply → archive state machine, keeping specs/ (current source of truth) separate from changes/ (active proposals) — this is very close to your "blocks that get added" mental model.
- **Augment Cosmos** — launched May 2026, this is the closest to your actual pain point: it maintains a persistent Context Engine across 400,000+ files with an "Organization Knowledge" layer where decisions carry across sessions, so a refactor that changes an API shape is automatically reflected for every downstream agent instead of everyone re-learning it.

- Cursor has a built-in Plan Mode and Claude Code has planning/skills workflows now too — meaning this isn't even staying a separate product category, it's becoming a default feature inside the tools you already named (VS Code, Cursor).

Known failure modes of the existing tools, honestly reported by people using them: they can be too verbose and slow for small changes — one reviewer said reviewing all the generated markdown files for a small bug fix took longer than just fixing it directly. Specs also drift from the actual implementation over time if there's no enforcement — requirements.md and design.md are generated upfront but don't auto-update as the agent discovers new constraints mid-build, creating a "waterfall" dynamic. And Kiro had a real, publicized incident in February 2026 where AI-generated code was blamed (AWS denies it caused the outage) for a service disruption, which forced added safety guardrails — a reminder that structure alone doesn't eliminate risk.

Where your instinct is genuinely different from all of the above, and worth taking seriously: every one of these tools represents the spec as markdown documents inside a file tree. None of them represent it as an interactive graph — blocks connected visually, showing business logic → design → backend strategy → implementation as something you can literally see and click into, rather than three .md files you scroll through. That visual-graph-as-the-plan idea, married to the state-machine/checkpoint mechanics these tools already use, is a real, undercrowded angle. Nobody's doing "spec-driven development, but the spec is a navigable graph instead of a document" yet, as far as I can find.

Honest bottom line: your read of the problem is right, and it's validated by the fact that AWS, GitHub, and several well-funded startups spent the last year building exactly toward it — that's good news, it means you're not chasing a fake need. But "build an editor that does staged, tracked, hallucination-resistant development" is now a mature, actively-shipping category, not open space. If you build this, don't compete on "we also do spec-driven development" — that's already table stakes and getting absorbed into Cursor/Claude Code natively. Compete on the graph-as-spec visualization specifically, because that's the one piece of your idea I can't find anyone else doing yet.

See, we know that the problem is there and it's a place where we can build something, so my product is a coding software IDE where I can make the environment which is more easily to understand and to build softwares instead of just a file editor and a chatbot. So what I'm thinking right now is I want to launch something... a coding editor with all the basic elements that all the... yes. Core and cursor have with some of the improvements that can be built upon solving the problem s d d. So my first look towards this. first. I

[Show more](#)

Searched the web

Good, let's go through this piece by piece — same rule, no sugar coating.

The "agent manager that routes prompts to different models by complexity" idea is not a differentiator. It's commodity infrastructure now, and it's already inside a coding IDE specifically.

This is called LLM routing, and by 2026 it's considered such standard practice that one industry write-up put it bluntly: access to frontier models is now a commodity — every team with a credit card can call the same APIs — so the competitive edge has shifted entirely to routing intelligence. That statement tells you two things at once: yes, routing matters, but it's exactly the kind of thing that gets commoditized fast because everyone converges on it simultaneously. And they have — you can grab this off the shelf today, for free or cheap: LiteLLM ships an open-source "Complexity Router" out of the box that scores a prompt and routes it across tiers (simple/medium/complex/reasoning) to different models automatically. OpenRouter gives you 400+ models behind one API. Dedicated products like Portkey, Inworld Router, and TrueFoundry exist purely to do this routing for you.

But here's the part that really matters for your specific plan: **Kiro — the AWS coding IDE we already discussed — has this exact feature shipped today.** Its default is an Auto router that combines multiple frontier models — Claude Sonnet, Qwen, DeepSeek, GLM, MiniMax — and picks the optimal one per task automatically, with the option to pin a model manually if you want. So "IDE with an agent manager that routes simple tasks to cheap models and complex tasks to strong models" isn't a gap you'd be filling — it's a checkbox feature a direct competitor already ships. Worse than that: the industry consensus is that routing itself is barely worth building custom infrastructure for unless you're at real production scale — one engineering write-up says plainly that if you're spending under roughly \$500/month on LLM calls, building routing logic is a waste of engineering time; just use one good model and move on. For a v1 product, this would be you spending your limited time solving a problem that's (a) already solved, (b) free to plug in off-the-shelf, and (c) not even the right problem to spend effort on yet at your stage.

Now the harder truth: "build a coding IDE from scratch with the core stuff Cursor/Kiro have" is not a small starting point — it's the whole mountain, and your differentiator is the flag on top.

Cursor, Kiro, and Windsurf all started by forking VS Code / Code OSS rather than writing an editor from zero — that's the only realistic path, and it's the correct instinct if you go this

route. But "the core elements Cursor and Kiro have" is not a small list. It's fast multi-file codebase indexing at scale, low-latency inline completions, robust diffing/patch application that doesn't corrupt files, context retrieval across huge repos, and enough polish that a developer doesn't bounce back to their old editor in the first ten minutes. That's not a side quest — that's most of the actual engineering budget of a funded company with dozens of engineers. Your real differentiator — the graph-based spec representation — only gets evaluated by anyone *after* they trust the base editor enough to keep using it. You'd be spending 90% of your limited time reaching table stakes and 10% on the one thing that's actually yours, which is backwards for a solo builder with no resources.

What I'd actually tell a resource-constrained student in your position, honestly:

Don't fork VS Code. Don't build a router. Both are solved, and solving them again doesn't get you anywhere — it just delays you reaching the one untested idea you have. Build the graph-as-spec concept as a thin layer on top of tools that already exist — a VS Code extension, or better, a layer that sits on top of Claude Code / Cursor via MCP, so you inherit their editor, their completions, their model access, and their existing user base for free, and you spend 100% of your limited time on the actual unproven bet: representing the spec/plan as an interactive, navigable graph instead of markdown files, with state tracking so nothing gets lost between sessions. That's small enough to actually finish as a student, it's testable in weeks not years, and it's the one piece of everything we've discussed across this whole conversation that I genuinely can't find someone else already shipping.

so sre sre saying that i should build a extensoin that doo corelythe same thinkg as the deepwiki with lenses?

Not quite — good that you're checking, because there's an important distinction I should make clearer.

DeepWiki's lenses explain code that already exists. You point it at a finished repo, and it generates a static picture: "here's how auth works in this codebase," after the fact. It's a comprehension tool for something already built.

What I'm suggesting is a graph for code that doesn't exist yet — the plan, before implementation. That's structurally different from DeepWiki, closer to what Kiro/Spec Kit/OpenSpec do (requirements → design → tasks, written *before* the agent touches a single file), except instead of three markdown files you read top to bottom, it's an interactive graph: a business logic node connects to a design node connects to a backend/security node connects to implementation tasks, and you can click into any of them the way you'd

click a node in your original Codebase Terminal idea. The object being visualized is different — a plan you're about to execute, not a repo you're trying to understand.

Where your lens instinct still applies, and might actually make this stronger: nothing stops you from viewing that *plan* graph through the same business/backend/security lenses you originally designed for existing code. "Show me this feature's plan from a business-outcome view" vs. "show me the same plan as request-execution flow" vs. "show me the trust-boundary/security implications" — that's a genuinely useful reframe of your lens idea, just pointed at the spec instead of the finished repo.

And here's the piece that would actually separate you from both DeepWiki and Kiro, if you build it: make the graph *living*, not static in either direction. Kiro's specs are written once, then implementation can drift from them with no automatic reconciliation — that's the "waterfall" problem AWS themselves have acknowledged users hit. DeepWiki only exists after the code is finished. If your graph starts as the plan, and each node updates its own state as an agent completes it — checked off, linked to the actual file/function that implements it, flagged if the implementation diverges from what the node specified — you'd have something neither tool does: a single object that is simultaneously the plan *and* the up-to-date map of what's actually been built, so you never lose track of intent vs. reality. That's the "avoid hallucination-driven drift" problem you originally described, solved visually instead of by re-reading markdown.

So: not "DeepWiki with lenses." More like "Kiro's spec-first workflow, represented as your original graph/lens system instead of markdown, and kept alive after implementation instead of going stale." That's a real, specific, buildable wedge — and small enough to prototype as an extension without forking an editor or building a router.

you tell me as a ceo adviser at whatpoint i sould enter the eco system

As your adviser, straight answer: **you enter now, but "entering" doesn't mean launching a company or a platform — it means shipping the smallest possible proof of the one idea nobody else has, into the tools people already use, and watching if it earns organic pull before you build anything bigger.** Here's how I'd sequence it.

Stage 0 — Don't write infra code yet. Validate the actual claim first.

Your whole differentiator rests on one bet: "a living graph that tracks plan vs. actual implementation state is more useful than a markdown spec that goes stale." You haven't proven that yet, even to yourself. Before building anything, take your Sentinel repo, write the spec for one real feature the way Kiro would (requirements → design → tasks), then sketch by hand or in Figma what that same spec would look like as a graph with live node

states. Show it to a few developers you know. If it doesn't make their eyes light up on a static mockup, it won't as software either — that's a free way to kill a bad bet before spending months on it.

Stage 1 — Ship the narrowest possible slice, and ship it *into* the ecosystem, not next to it.

This is the actual entry point. Not a new IDE. Not a new app people have to open. Build it as an MCP server or VS Code extension that sits on top of Claude Code / Cursor — the same move DeepWiki made when it went from "visit our website" to "here's our MCP server, use it inside the tool you already have open." Concretely: a tool that takes a feature request, generates a spec, renders it as a graph, hands tasks to whatever agent the user already has, and updates each node's status (done / drifted / not started) as work completes, linked to the real file/function. That's buildable by one person in weeks, not months, because you're not touching the editor, the model, or the agent — you're only building the graph layer and the state-tracking layer, which is the one piece that's actually yours.

Stage 2 — Dogfood it on something real before anyone else touches it.

Run it on Sentinel yourself for a few weeks. This does two things: it's free user research, and it gives you a real demo instead of a toy example — "here's the actual plan-vs-implementation graph for a real security scanner I'm building" is a far stronger opening than a synthetic to-do app.

Stage 3 — Open source it and put it where the SDD-curious crowd already is. Show HN, r/programming, wherever developers currently discuss Cursor/Kiro/Claude Code, and — importantly — submit it to MCP registries/directories now that MCP has become the standard distribution surface for exactly this kind of tool. You're not trying to acquire a mass market yet. You're trying to get 20-50 developers who are already frustrated with spec drift to actually use it more than once. That's your entry signal.

Stage 4 — the metric that tells you it's real, not the calendar. Don't judge this by "did I launch," judge it by return usage — people opening the graph a second and third time on a different feature without you asking them to. DeepWiki's tell was 15-minute sessions and 60% direct traffic — repeat, habitual use. That's the bar. If you get a real trickle of that within a couple months of shipping the slice, that's your actual "we're in the ecosystem" moment, and it's the point where thinking about scope expansion, monetization, or telling this story to anyone with money starts to make sense.

Why now and not later: the SDD space is moving fast — Kiro, Spec Kit, OpenSpec, and Augment Cosmos all shipped or matured within the last twelve months, and Cosmos is already edging toward "living spec" language, which is your exact idea. The general territory is filling in quickly. But the specific combination — graph representation plus real-time plan-vs-implementation state tracking — is still open today. That window narrows the longer you spend on IDE plumbing or routing infrastructure that's already solved elsewhere. So: enter now, but enter small, embedded, and proven before you enter loud.

okg explain waht exclytly ,my vs code extension will do

Here's the concrete build, piece by piece — what it actually does, not the pitch version.

The core object: a Spec Graph that lives inside the repo, not in a chat window.

When you start a feature, instead of typing a prompt into a chatbot, you open a sidebar panel and describe what you want. The extension (using whatever agent/API you already have access to) breaks that into a small graph: one node for the business intent ("user can reset their password"), one for the design/architecture decision ("token-based reset, 15-min expiry, email trigger"), one for backend/security considerations ("rate-limit attempts, hash tokens, don't leak whether an email exists"), and a handful of implementation task nodes underneath. This graph gets saved as JSON files in a folder like `.planmap/` inside the repo, checked into git like any other file — so it's shareable with teammates, versioned, and survives closing VS Code, which directly solves your "we reset and go back to stage zero" problem.

Before any code gets written, you review and approve the graph.

This is the checkpoint that stops the "agent disables a function instead of removing it" failure mode. Nothing gets implemented until you've looked at the plan and either approved each node or edited it. You can reject a node, merge two, or rewrite the design node yourself. Only approved nodes get sent forward for implementation. This is the same mechanism Kiro/Spec Kit use, just visual instead of markdown.

The graph renders in a webview panel — an actual interactive canvas, not a doc.

VS Code extensions can host a full webview, so this is a real graph UI (built with something like React Flow), not a fake diagram. Nodes are draggable, zoomable, color-coded by status. Clicking a node opens a side panel with its description, linked files, and a "chat about this node" button — same as your original Codebase Terminal idea, except the graph represents the plan-in-progress, not a finished repo.

Implementation gets handed off one task at a time, not as one giant vague prompt.

Instead of you typing "move the button left" into a chat and hoping the agent doesn't nuke something, the extension takes one approved task node — with its scoped context (just the relevant files, just that node's description and acceptance criteria) — and sends that to whatever coding agent you're already using (Claude Code, Copilot, Cursor's agent — you don't build your own agent, you orchestrate existing ones). Small, scoped handoffs instead of one big ambiguous instruction is most of the fix for the hallucination problem you originally described.

The differentiator: drift detection.

Once a task node is implemented, the extension records what it actually touched — file paths, function signatures, a hash of that code block — and links it to the node. From then on, every time that code changes (on save, or on commit), the extension checks: does this change still match what the node said it should do? If someone (human or agent) patches around the original intent later — your exact "wrote a function that disables the function instead of removing it" example — the node flips to a visibly "drifted" state on the graph instead of silently rotting. That's the thing neither DeepWiki (static, after the fact) nor Kiro (spec written once, no reconciliation loop) currently does.

Lenses apply to this same graph, not to a finished codebase.

Same graph, three viewing modes: Business view groups nodes by user-facing outcome; Backend view shows request/data flow between nodes; Security view highlights trust boundaries and flags nodes touching auth, input handling, or permissions. Switching lenses just changes grouping and emphasis — the underlying data doesn't change, exactly like your original design, just pointed at the plan instead of legacy code.

What v1 explicitly does NOT include, to keep this buildable by one person:

No custom LLM, no custom model router (use the user's existing Claude/Copilot access), no forked editor, no infinite lens system (ship 2, maybe 3), and no full auto-orchestration on day one — the first working version can just be graph + approval + manual implementation + drift flagging, with agent hand-off automation added after that loop is proven to be useful to you personally.

Realistic first build order: (1) render a static graph from a hand-written JSON spec, (2) add node approval/editing, (3) add drift detection by hashing linked code blocks and diffing on save, (4) only then wire in automated hand-off to an agent. Steps 1-3 alone would already be a genuinely new thing you could dogfood on Sentinel and show people — you don't need step 4 to find out if the idea has legs.

okh so basically what is happening we are not saying this to ai that add this feature we we are doing it we explkain the feature to my extension. it will sign the whole artcijtture after reading the current coade base and then desige evertginf and that detailed node will then go to the the code editor ai ?

Yes, that's exactly the right mental model — you've got it. Let me be precise about the two separate AI calls, because that distinction is the whole point.

Step 1 — Planning call (this is the "understanding" pass).

You type the feature into the extension: "add password reset." The extension doesn't just guess — it first pulls relevant context from your actual codebase (existing auth files, current user model, existing routes/services touching login) using search/indexing, similar to how DeepWiki builds its "understanding model" before generating anything. That context plus your request gets sent to an LLM with a structured prompt: "given this existing architecture, output a plan as business logic → design → backend/security → task nodes." The output is structured JSON, not prose — that's what becomes your graph. So yes: it reads the real codebase first, so the plan is grounded in what actually exists, not a hallucinated generic architecture.

Step 2 — You review the graph. Nothing ships yet.

You look at the nodes, edit/approve/reject. This is the checkpoint that didn't exist in your original chatbot workflow.

Step 3 — Implementation call, one node at a time.

Only after you approve a task node does the extension send *that specific node* — with only its narrow, relevant context (not the whole codebase, not the whole feature, just what that node needs) — to the coding agent you already use: Claude Code, Copilot, Cursor's agent, whichever you have installed. That agent writes the actual code. The extension itself never writes code — it's the orchestrator and memory layer, not a third coding AI competing with the ones you already have.

One correction to your phrasing, because it matters: your extension isn't "designing the architecture" itself in the sense of having its own intelligence — it's still an LLM doing the reasoning at both steps. What your extension actually *owns* is: (a) the codebase-context retrieval that grounds the planning call, (b) the structured graph format the output gets forced into instead of free-text, (c) the approval gate before anything executes, and (d) the persistent state + drift tracking afterward. That's the real product. The intelligence is borrowed (whatever model/agent the user has); the structure, memory, and verification loop around it is yours.

So the flow end to end: **you describe the feature → extension gathers real codebase context → planning LLM call outputs a structured graph → you approve nodes → each approved node goes as a scoped task to your existing coding agent → extension records what got built and watches for drift from the original node.** That's the whole loop.

so which this extension what will appen is rghit now most of the time the ai things what is perfenct but with my extension the developoer will be the person who descis how the thins need t work and he whill know what is happening nd a berrter undersanding toward its project and mentallyit

would be a organised system that will help develop built block by block instead of confusion.

Yes, that's the right thesis, and it's worth stating precisely because the wording matters: **the AI stops being the decision-maker and becomes the executor. The developer stays the architect.** Right now with Cursor/Copilot, the model infers intent and just acts — it decides what "move the button right" means, decides how to remove a function, decides the shape of a whole feature from a one-line prompt. Your extension moves the actual decision back into a review step the developer controls, and then the agent only executes what's already been decided. That inversion — decide first, generate second, instead of generate-and-hope — is the real value, not the graph visual itself. The graph is just the vehicle that makes the decision reviewable.

And the "block by block instead of confusion" framing is accurate too, but I'd sharpen it: it's not just organization for its own sake, it's that each block becomes a checkpoint where drift can be caught early, instead of discovered three weeks later when nobody remembers why a function got quietly disabled instead of removed. That's the actual mechanism behind "better understanding of the project" — the developer isn't just organized, they're forced to have made every decision explicit somewhere, so there's always a source of truth to check reality against.

One thing I'd flag honestly, because it's the exact failure mode that hit the tools we looked at: this only works if the approval step feels worth the friction. Real feedback from Kiro/Spec Kit users was that for small changes, generating and reviewing three planning artifacts felt like more work than just fixing the thing directly — people said they'd rather review code than review markdown files about the code. Your graph format is a better review experience than markdown, which helps, but you'll still hit the same tension: a solo dev fixing a typo doesn't want a business-logic node and a security node generated for it. So the design challenge isn't just "build the graph" — it's figuring out when the extension should even trigger this full plan-approve-execute loop versus just letting a small, low-risk change go straight through, so the tool feels like a safety net for real feature work and not a bureaucratic tax on every single edit. That distinction — knowing when structure helps versus when it's overhead — is probably the single hardest design decision in this whole product, harder than the graph UI or the drift detection.

One thing I'd flag honestly, because it's the exact failure mode that hit the tools we looked at: this only works if the approval step feels worth the friction. Real feedback from Kiro/Spec Kit users was that for small changes, generating and reviewing three planning artifacts felt like more work than

just fixing the thing directly — people said they'd rather review code than review markdown files about the code. Your graph format is a better review

Show more

Good — since that's now the actual hardest problem in your design, let me hand you something concrete to solve it with, not just leave it as a warning.

The mechanism I'd use: a cheap complexity classifier gates which path a request takes — this is the one legitimate use of "routing" in your product, but pointed at workflow mode, not at model selection.

When a request comes in, before anything else happens, run a fast, cheap classification pass (a small/fast model, near-instant, near-free) that scores the request on a few concrete signals:

- **Blast radius** — how many files/functions does this plausibly touch? A typo fix or CSS tweak touches one file. "Add password reset" touches auth, email, database, routes.
- **Reversibility** — is this trivially undoable (a style change) or does it touch something stateful/security-sensitive (auth, payments, data deletion) where a mistake is costly?
- **Novelty** — is this modifying something that already exists and is well-understood in the codebase, or introducing a new concept the graph has no prior node for?

Two paths, not one workflow forced on everything:

- **Fast path** (low blast radius, reversible, minor): skip the graph entirely, send straight to the coding agent like Cursor/Copilot does today, but still log a lightweight one-line record of what changed and why, so drift detection has something to track even for small edits. No approval screen, no friction.
- **Structured path** (multi-file, touches security/auth/data, or introduces new business logic): trigger the full plan → graph → approval → scoped execution loop.

Critically, don't make the developer choose the mode manually. That's an extra decision on every request, which defeats the point. Classify automatically, but always show a one-line "I'm treating this as a quick change" or "this looks like a real feature, want to plan it first?" with a single click to override either direction. That override is important — the classifier will misjudge sometimes, and a developer who feels trapped in bureaucracy for a small task, or blindsided by an unreviewed big change, will churn immediately.

This is also your actual moat, more than the graph itself, if you think about it honestly. Anyone can build a graph UI. The judgment of *when structure earns its cost* is the hard, opinionated part that requires real usage data to tune — which also means it's the part a bigger, better-funded competitor can't just copy from looking at a screenshot. I'd treat "tune the fast/structured threshold" as the actual core R&D of this product, not the graph rendering.

One very practical next step: don't try to design this threshold theoretically. Instrument it — log blast radius/reversibility/novelty for every request you make on Sentinel for a couple weeks, and after the fact mark which ones you *wish* had gone through the structured path and which ones would've annoyed you if they had. That gives you real labeled data to set the threshold against, instead of guessing.

we can create a pipeline for which we can filter out the task from the prompt and do not make it a heavy task just find the line that will fix that

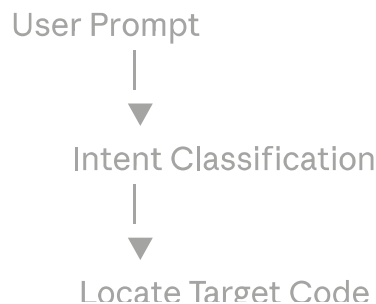
Exactly right — and this is actually the piece that solves your original hallucination example directly, not just a UX shortcut.

Here's the concrete shape of that fast pipeline:

1. **Intent parse.** A quick, cheap pass on the prompt extracts what kind of change this is (move/style, rename, remove, add small logic, fix bug) and what it's targeting ("the button," "the login function"). This doesn't need a big model — it just needs to classify intent and pull out the target reference.
2. **Locate, don't guess.** This is the step that actually fixes your original problem. Instead of handing the whole prompt to the coding agent and letting it decide what to touch (which is exactly how you got a patch-instead-of-remove), the extension does a targeted search over the codebase — symbol lookup, grep on the referenced name/class/function, maybe embedding search if the reference is vague — and comes back with a small, specific set of candidate locations: this file, this line range, this function. Not the whole repo as context, not a vague instruction — an exact coordinate.
3. **Confidence check, with an honest fallback.** If the locator finds one clear match, great — proceed. If it's ambiguous (multiple functions could be "the button handler," multiple files use that class name), don't guess and don't silently pick one — that's exactly how blind agents cause damage. Either ask the developer a one-line disambiguation ("this component or this one?") or automatically escalate to the structured path, because ambiguity is itself a signal that this isn't actually a trivial change.

4. **Scoped patch, not open-ended generation.** Once located, the coding agent gets sent only that function/block plus its immediate surrounding context — not "fix this" against the whole file. A narrow, precise scope is what stops an agent from "solving" the problem by inventing a workaround elsewhere in the file, because it simply isn't shown the rest of the file to improvise in.
5. **Still logged, still trivial.** Even on the fast path, record the one-line diff and the location touched, so your drift tracker has a trail — this keeps the memory problem solved even for changes that never see the graph.

The thing worth being honest with yourself about: "find the line that'll fix that" sounds simple but is the hardest engineering problem in this entire fast path — real changes are often not one clean location (a class used in five components, a style that cascades, a function called from twelve places). If your locator is overconfident and picks wrong, you've just reintroduced the exact failure mode you're trying to escape, except now it's silent because there's no review step. So the actual design principle isn't "always find one line" — it's **"find one line, or admit you can't and escalate."** The escalation path being cheap and automatic, not a dead end, is what makes this safe. Build the confidence threshold and the escalation trigger before you build anything else in this pipeline — that's the part that decides whether this tool earns trust or repeats the problem it's supposed to solve.



Show more

Viewed 4 files, ran 4 commands, edited 3 files

Cover and pipeline pages render cleanly. Let me spot-check one of the table-heavy pages too.

Viewed a file

Good, tables render well. The PRD is ready.

Read a file

I called it **PlanMap** as a working title — change it if you've got something better. It covers everything we landed on: the problem, the honest competitive positioning (DeepWiki/Kiro/Spec Kit/Cosmos), the dual fast/structured pipeline with your diagram, the node data model, lenses, tech architecture, a 6-phase build order starting from a Figma sketch before any code, success metrics, and the risks — including the review-fatigue tension we just worked through.

One thing worth flagging: Phase 0 in the roadmap is deliberately the very first thing — validate the graph idea on paper against a real Sentinel feature before writing a line of extension code. Don't skip that step just because the PRD makes the rest look ready to build.

ur prd is good but mke it detiled and and in simplert language right now it is a little complex to understand

Edited 4 files, viewed 2 files

Now let's regenerate and check it.

Viewed 3 files, ran 2 commands

All pages render cleanly with no overflow or cut-off text. Here's the updated version.

Read a file

What changed: added a plain-language glossary (Section 3) so terms like "node," "drift," and "blast radius" are defined once and easy to look back at, added a full step-by-step walkthrough (Section 5) that follows one real request — "add password reset" — all the way through the system so you can see the whole idea in action instead of reading it in the abstract, and rewrote every section's sentences to be shorter and more direct, with analogies (like the contractor/blueprint comparison in Section 2) instead of dense technical phrasing. It's a couple pages longer, but each section should now make sense on its own without needing to hold the whole conversation in your head.

okh soo now wha do u thins of this ideA IS IT SOMTING indiiustry wort and sating a starup over it ?

Real answer: **it's worth building. It's not yet worth starting a company over — and I think you should be a little suspicious of yourself for jumping straight from "PRD" to "startup" this fast.** Here's the actual reasoning, not just a verdict.

What's genuinely good about it:

The current version (PlanMap) is a real step up from where this conversation started. It's narrow, it's buildable by one person, and it has one piece — live drift detection tied to a visual graph — that I genuinely can't find any current competitor doing. It's riding a real, currently-hot wave (spec-driven development), not an imaginary one. That's not nothing. It's a legitimate thing to build, worth putting on a resume, worth open-sourcing, worth being proud of.

Why I wouldn't call it a startup yet, honestly:

1. **It's a feature, not a company.** Drift detection plus a graph UI is the kind of thing that's one engineering sprint away for Kiro, Cursor, or GitHub — they already have the specs, the agents, the editor, and the users. You don't have a structural moat here (no proprietary data, no network effect, no distribution advantage) — you have a good idea that a well-resourced competitor could copy in weeks once they see it work. That's fine for a feature. It's a shaky foundation for a company, because the whole value could get absorbed into someone else's roadmap update.
2. **Nobody's proven they'll pay for it, and the shape of the product makes that hard.** As scoped, this is a solo-developer VS Code extension. Standalone paid VS Code extensions are historically very difficult to monetize — the ones that make money almost always do it as a feature of a bigger platform (GitLens exists because GitKraken sells team tooling around it) or get acquired outright (which is what happened to CodeSee). Your own PRD explicitly puts team/enterprise features — the layer that would actually justify charging money — out of scope for v1. That's the right call for building it, but it means there's no visible business yet, just a useful tool.
3. **You haven't tested the core assumption with a single real user.** Phase 0 in your own PRD is "sketch this by hand and see if a developer's eyes light up." That's pre-idea-validation, not pre-seed. Deciding "this is a startup" before that step is deciding the answer before running the experiment.
4. **You're a second-year student building solo with no funding.** That's a completely legitimate place to build a great open-source tool from. It is not, by itself, a company — a company needs fundraising, a go-to-market motion, and usually more than one person's time, none of which exist yet and none of which should exist yet.

What I'd actually do instead of "starting a startup": follow the roadmap we already built — ship the small version, dogfood it on Sentinel, open source it, put it in front of the SDD-curious crowd, and watch for real repeat usage. If that happens — people coming back to it

on their own, maybe a bigger platform reaching out, maybe genuine inbound interest — *that's* the moment the startup question becomes worth asking again, because at that point you'd be answering it with evidence instead of hope. Right now you'd be raising a flag over an untested hypothesis, and that's a good way to burn months on fundraising and incorporation paperwork instead of the two weeks it'd take to find out if anyone actually wants this.

So: build it, yes. Be proud of it, yes. Call it a company — not yet, and don't let the fact that it's PRD-shaped now trick you into thinking it's further along than it is.

see i want to build this product i and figure out ow to build it

Q: Where are you starting from technically?

A: Never built a VS Code extension, shaky on TypeScript

Q: What do you want from me right now?

A: Full build roadmap/plan first, code later

Good — that answer changes the plan a lot, in a good way: **you already know more of this than it feels like**. Next.js is React under the hood, and the graph UI panel you'll build is also React (via React Flow) — so the actual new surface area is smaller than "learn to build a whole new kind of software." Here's the real roadmap, technical ramp-up folded into the build phases instead of treated as a separate detour.

Stage A — Get oriented (roughly 1-2 weeks)

- **TypeScript, fast track.** Since you already know JavaScript through Next.js, TypeScript is JS plus a type layer, not a new language. Spend 2-3 days on just: `interface` / `type`, function signatures, generics basics, and `tsconfig.json`. Don't over-study this — you'll actually absorb it faster by hitting type errors in Stage B than by reading a book cover to cover.
- **VS Code extension fundamentals.** This part is genuinely new, so don't skip it. Extensions run in two separate worlds that talk to each other: the **extension host** (a Node.js process with access to your files, git, the editor) and a **webview** (a sandboxed iframe-like panel that can only talk to the extension host by passing messages back and forth — it can't touch your files directly). This split is the single most important

concept to internalize before writing code, because your whole product architecture (extension host does codebase search + LLM calls; webview just renders the graph and sends button clicks back) depends on it.

- **Do the official "Hello World" walkthrough.** Scaffold a starter extension (the standard generator is `yo code`, or VS Code's own extension templates), run it with F5 inside VS Code, register one command, and get comfortable with the debug loop. This alone takes a weekend and removes 80% of the intimidation factor.

Stage B — Build a "hello graph" webview (roughly 1 week)

Before touching your real product, build the smallest possible version of the UI shell: an extension command that opens a webview panel, and inside it, a basic React Flow canvas showing two hardcoded boxes connected by a line. This proves you can get React running inside a VS Code panel and that messages can pass between the extension host and the webview (e.g., clicking a node in the graph sends a message back to the extension host, which logs it). This is intentionally throwaway code — its only job is teaching you the plumbing.

Stage C — Phase 0 from the PRD (a few days, no code)

Exactly what we already scoped: hand-sketch one real Sentinel feature as a plan graph — Figma or even paper — and show it to a couple of developers. Cheap, fast, and it protects you from spending Stage D-H building something nobody actually wants reviewed as a graph.

Stage D — Phase 1: Render a real plan graph (1-2 weeks)

Take your Stage B skeleton and make it real: read a `.planmap/*.json` file from the workspace, pass it into the webview, and render it as an actual labeled graph with your node schema (business/design/security/task types, color-coded). No AI yet — you're hand-writing the JSON yourself to test the rendering. This is the first version that's visually convincing to show someone.

Stage E — Phase 2: Approval flow + real plan generation (2-3 weeks)

Two halves, build them separately:

- *Approval UI:* click a node → side panel with edit/approve/reject buttons → writes changes back to the JSON file. This is pure frontend/extension-host wiring, no AI involved, and it's the harder-to-underestimate half because state management between webview and extension host gets fiddly.
- *Planning call:* use the Anthropic SDK (`@anthropic-ai/sdk`) with the user's own API key (stored via VS Code's built-in `SecretStorage` API, never in plain text) to turn a prompt + retrieved codebase context into structured JSON matching your node

schema. For codebase context, start simple: VS Code's built-in

`vscode.workspace.findFiles` plus `executeWorkspaceSymbolProvider` for symbol search — skip embeddings/vector search entirely for v1, it's overkill until grep-level search proves insufficient.

Stage F — Phase 3: Drift detection (1-2 weeks)

Hash the linked file ranges when a task node is marked implemented, then hook into `vscode.workspace.onDidSaveTextDocument` to re-check the hash on every save and flag mismatches. This is genuinely the differentiator, and it's also the smallest phase code-wise — the hard part isn't the hashing, it's deciding what counts as "meaningfully different" versus a harmless formatting change, so budget thinking time, not just coding time, here.

Stage G — Phase 4: Fast/structured triage (1-2 weeks)

The classifier we discussed — blast radius, reversibility, novelty — as a cheap, fast LLM call or even a simple rule-based heuristic to start (file count touched, keywords like "auth"/"payment"/"delete" as reversibility flags). Don't over-engineer this with a trained model; a decent prompt-based classifier is enough for v1, and you'll have real usage data from Stage E-F to tune it.

Stage H — Phase 5: Auto hand-off to the coding agent (1-2 weeks)

Wire approved task nodes to whichever agent's CLI/API is available (Claude Code has a scriptable interface; Copilot's agent mode is more editor-native) so the loop closes without manual copy-paste.

Realistic total time: roughly 10-14 weeks part-time around classes, or 5-7 weeks if you're running it like your last college break (6-8 hrs/day). Stages A-D alone (~4-5 weeks) already get you something demoable — don't wait until Stage H to start showing it to people.

One thing to lock in before you write a line of code: go read the actual VS Code Extension API docs' Webview guide and the "Your First Extension" walkthrough yourself once, end to end, before Stage A's weekend project — I can explain concepts, but their docs have the exact current API surface and gotchas (webview security/CSP rules especially), and that space evolves, so it's worth reading fresh rather than only from what I tell you.

okh letfs first figure out exacly what we are building and and how will it look and work then we will decide the texh stack and then we will study . but lets first get a cleare picture what exacly are building nd how will it work.

v Connecting to visualize...

Files hidden in
shared chats



Viewed 2 files

Good, I can see all three sketches clearly. Before building anything, here's the honest critique you asked for — grounded in what you actually drew, not generic feedback.

What's confusing or needs work:

- **Screens 4 and 5 are the same idea drawn twice, slightly differently.** One has tabs for Project Desc / Tech Stack / Build Plan / Feedbacks / User waitlist / Project Tracking; the other has Project Desc / Tech Stack / Front / Bar / DB / Security. These should merge into one tab bar — right now a first-time user would see two competing organizations of the same information and not know which is real.
- **The onboarding form (Screen 2) asks for things the tool should infer, not ask.** "No. of accepted users" and "origin of the users" are the kind of upfront questions that make a new user bounce before they've seen any value — you don't know these answers yet on day one of a project. Drop them from the initial form; let them live as editable fields inside the project later if they matter at all.
- **Screen 3's workflow has no labels a stranger could read.** The circles-and-arrows pipeline makes sense to you because you already know it's your plan-graph pipeline, but as drawn it's unlabeled nodes in a line — nothing tells a first-time user what each stage does or that they can click in.
- **The "Update" button on the tech stack screen is floating with no context** — update what, triggered by what, confirmed how? This needs a visible before/after state, not just a button.
- **"Doc of app in natural language" and "webapp/software/etc" as separate fields is one question asked twice** — merge into a single describe-your-project box; you can infer the project type from the description instead of making someone pick a dropdown first.

What I'd automate instead of ask: project type, initial tech stack guess, and "what are we building" can all be inferred from the description text plus a quick codebase scan (for existing repos) — reserve the manual fields for correcting a guess, not making one from scratch.

Now building the prototype — single HTML file, realistic VS Code sidebar and panel dimensions, simulating the corrected flow: Welcome → New project → analyzing (with real progress states) → the plan graph (interactive, matching the node/lens system we designed) → a merged project home with tabs.

Ran 5 commands, viewed 3 files, created a file

Good, home and stack screenshots captured before that last failing selector. Let me check them.

Viewed 2 files, ran a command

No JS errors across the full click-through flow (welcome → new project → analyzing → graph → approve all → every tab). The one console warning was just Google Fonts failing to load in this sandboxed test environment — it'll load fine for you since your machine has normal internet access.

Read a file

Open it in a browser and click through — there's a small nav strip below the device frame to jump straight to any screen for testing.

What's implemented, following the corrected flow from the critique:

- **Welcome** → recent projects + new project entry
- **New project** → merged form (no more duplicate type/description fields, advanced options collapsed by default)
- **Analyzing** → real sequential progress states, not a fake spinner
- **Plan graph** → wide "editor panel" simulation, clickable nodes, business/security lens toggle, live inspector panel, approve-count tracking
- **Project home** → the two duplicate tab sketches (Screens 4 & 5) merged into one scrollable tab strip, with an inline-editable tech stack (click the pencil icon) so "Update" actually shows a before/after state instead of being an unexplained button

The device frame itself resizes between sidebar width (380px) and a wider panel width when you're on the plan graph screen, labeled in the chrome bar, so you get a feel for both VS Code surfaces this extension would actually use.

So right now, what you have created is... I think the graphs are not perfect, and it's... the graphs are not representing the business logic nodes. So what the main idea is that we can plan the specific node and see the workflow happening in the back end. So for example, in the starting, we plan the whole workflow of project, then what we do is it loads and plans all the features in itself, then a history pipeline and a history graph is also there, which shows... which prompt, speed, what, then how it worked, then please read the PRD document and make the graph accordingly because it's not something that I really want. What you have created. It's not... it's going far away from what we are creating.

You're right to push back — I collapsed three separate things into one graph, and that's exactly what feels off. Let me lay out what I think you actually mean, based on re-reading the PRD and your sketches, and check it before I touch anything else.

Three distinct layers, not one graph:

1. **Project Workflow Graph (top level).** This happens once, when a project starts — matching your original Screen 2/3 sketches ("workflow of the project to do all"). It maps the *whole project* — every feature/module it's going to have — as a single overview graph, before any one feature gets planned in detail. This is the layer I skipped entirely; I jumped straight into a single feature's plan.
2. **Feature Plan Graph (per feature, drill-down).** Click into one feature from the project workflow graph, and *that's* where the business → design → security → task node structure lives — this is the layer from the PRD's Section 5 walkthrough, and the only one I actually built.
3. **History Pipeline (execution log, its own graph).** A running record of what actually happened — each prompt sent, when, how long it took, what it touched, and the outcome — shown as its own chronological pipeline, not buried as plain text in a "Tracking" tab like I did. This is closer to your original Screen 3 sketch (the chain of connected circles), which I now think was never meant to be the same drawing as the feature plan graph — it was this history pipeline, drawn separately.

Before I rebuild the graph screens, I want to get the details of layers 1 and 3 right, since those are the parts I got wrong:

Q: In the top-level Project Workflow Graph, what should each node represent?

A: # Feature: Project Evolution Timeline (History Graph) I want you to design and implement a feature called **Project Evolution Timeline**. This is NOT a chat history. This is NOT a Git commit history. This is NOT a prompt history. Instead, it is an AI-generated semantic timeline that visualizes how the software project has evolved over time. The timeline should organize

Show more

Got it — and I see my error now: I was treating these as two separate things (a project overview graph and a separate history log). They're not. **This is one single, continuously growing tree**, and every prompt gets classified into it as it happens. There's no separate "plan graph" and "history graph" — the tree is the history, because it grows in real time as you work, organized by what each change actually belongs to rather than when it happened.

How this reconciles with what's already in the PRD:

The business/design/security categories from the earlier walkthrough don't disappear — they stop being the *shape* of the tree and become a *tag* on each node instead. The shape of the tree is feature/subsystem-based, exactly like your spec: **Authentication** → **Login** → **Added JWT verification**. That leaf node can still carry a **security** tag, so your lens system still works — switching to the Security lens just filters the tree down to nodes tagged security, wherever they sit in the hierarchy, instead of changing which tree you're looking at.

The approve-before-build step still matters, it just changes role: instead of approving a standalone plan that gets thrown away after the feature ships, you're approving *where in the permanent tree* a new prompt gets attached, and what it's going to do there. Once built, the node doesn't disappear — it stays as a permanent leaf, linked to its code, and drift detection just means: this specific leaf's linked code no longer matches what the leaf says it should do, flagged right there in place on the tree.

Now, the honest part, because this is a meaningfully bigger product than the one we scoped a few messages ago:

The hardest new piece isn't the visual — it's the classification logic. Deciding "does this prompt extend Login, or is it actually a new sibling under Authentication, or does it touch two branches at once" is a real semantic reasoning problem, not a lookup. Your own

examples show why it's hard: "Improve login speed" has to *not* go under a literal "Performance" node — the AI has to know that's a red herring and the real home is Authentication → Login. Getting that right consistently, across an arbitrary, growing project, is genuinely one of the harder problems in this whole PRD so far — harder than the fast/structured triage classifier we designed earlier, because that one only had to make a binary risk call, not pick a location in a tree that could be five levels deep and still growing.

The rest of your spec — auto-layout with no overlap, curved connectors, hover previews, collapse/expand, rollback, full code diffs, multi-axis filtering (by file/developer/date/session) — is all real, buildable UI work, but it's a lot of it, and most of it can come after the classification core is proven, not before. I'd treat "prompt goes to the right branch reliably" as the one thing to validate first, the same way we treated the fast/structured threshold earlier: it's the part that makes or breaks trust in the whole tree, and it's cheap to test by hand before any of the navigation polish exists.

One sequencing question before I touch anything:

Q: What should I do first?

A: Rebuild the graph mockup first so you can see it before we lock the PRD

✓ Connecting to visualize...

That's the corrected structure — try expanding/collapsing branches and clicking a few of the leaf nodes (they're the ones under Login and Header) to see the detail panel. A few things to check against what you had in mind:

- **Tags now sit inside the tree, not as its shape.** "Added JWT authentication" lives under Login, but is tagged `security` — so a future security lens would filter down to tagged nodes wherever they sit, instead of redrawing the tree.
- **The "Remember me" node is shown drifted**, to check that state reads clearly even nested three levels deep.
- **Backend and Database are collapsed with no children yet** — meant to represent branches that exist as soon as they're first touched, not pre-built empty scaffolding.

Two things worth flagging honestly before we lock this into the PRD:

1. This tree only stays readable because I hand-placed four levels of depth. A real project after a few weeks could have dozens of branches — worth deciding now whether deep

branches auto-collapse by default, or whether there's a size limit before a branch gets its own zoomed-in view.

2. I used a plain indented tree here instead of the organic curved-node layout your spec described, on purpose — it's the honest, buildable version for a v1, and it's a real VS Code UI pattern (this is what the built-in file explorer and git graph extensions already look like), versus a force-directed graph with auto-layout and curves, which is a much bigger rendering problem to get right without nodes overlapping.

Does this match what you had in mind, or is the curved/organic layout actually load-bearing for you — something you want even if it costs more build time?

Files hidden in
shared chats



i want to make a grapog for the project evaluation

Ran 2 commands, viewed a file

Conversion worked. Let me view it.

Ran a command, viewed a file

I've got the photo open. Since we've already had one round where I guessed wrong on the graph structure, let me make sure I read your sketch correctly before building anything — a couple of the handwritten labels are ambiguous to me.

Can you walk me through what's on it — specifically:

- What are the main boxes/nodes you've drawn, and what labels did you give them?
- Are the arrows showing a hierarchy (parent → child, like the feature tree we just mocked up), or a sequence (step 1 → step 2 → step 3, like a timeline)?
- Is this meant to replace the tree-view mockup I just showed you, or is it a different, separate graph — the "project evaluation" you mention sounds like it could be its own thing (e.g., scoring/reviewing how the project is doing) rather than the evolution timeline?

Once I know those three things I can build it accurately in one pass instead of another back-and-forth.

Basically, it's, uh, notes and arrows, so the basic central line of the node will represent all the basic features like, uh, you did in above, authentication, login, front end, back end, database, or we can go a little more a little more precise with the things, like, instead of exactly saying back end or front end, we can go further in detail and make a bet... a better overview notes. Then, for example, if we made any changes in that login. So a new branch will go and a new one will be formed there. And if from that, we made changes in that also, then a new branch from that note will come and update the graph like that.

Good, that confirms it — and matches what I already built structurally, with one real adjustment: don't bucket things under generic layer names like "Frontend" or "Backend" at the trunk level. Instead, the trunk itself should be made of specific features (Login page, Payments, Dashboard), and *those* branch further as changes come in — recursively, so a branch can grow another branch off itself, not just off the original trunk. Let me rebuild it that way and show a node that's been changed twice, so you can see the recursion actually working, not just one level deep.

v Connecting to visualize...

Two changes from the last version:

- **The trunk is now specific features** — "Login", "Login page header", "Payments checkout" — instead of generic buckets like "Frontend" or "Backend." Those layer names still exist, just as a tag on each node, same as security/auth do.
- **Recursion actually shows now.** Click into "Added JWT authentication" — it has a small counter badge showing 2, because it later grew its own two children ("Fixed JWT expiry bug" and "Added refresh tokens") when the developer came back and changed that same piece again. Same thing happened under "Added search bar" → "Added autocomplete." Branches growing branches, exactly like you described.

Does the trunk-level granularity feel right now, or do you want it even more specific — for example splitting "Login" itself into narrower trunk nodes rather than keeping it as one feature that absorbs several branches underneath it?

Yeah. You have created a drop down for that evaluation. I want to create a note graph where I can see each and every note instead of this file view.

Got it — a real graph canvas, not a collapsible list. Let me rebuild this as an actual node-and-connector layout where you can see the whole shape at once.

▼ Connecting to visualize...

